Elementary data structures

Part III Data Structures

Scope

The difficult part of elementary data structures is often the representation and the operation's preconditions. Physical array order is not queue order, a node handle is not a key, and a free slot is not necessarily cleared. Concrete before-and-after states establish the link invariants and costs of queues, lists, allocators, and tree representations.

Prerequisites

Arrays, pointers, abstract data types, and asymptotic analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Represent stacks and queues with arrays or links.
- Maintain linked-list invariants and boundary conditions.
- Represent objects with index links when native pointers are unavailable.
- 10.1 Stacks and queues
- 10.2 Linked lists
- 10.3 Implementing pointers and objects
- 10.4 Representing rooted trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stacks and queues

An abstract data type (ADT) specifies operation semantics before choosing a representation. A stack's pop removes the most recently inserted element that remains, giving LIFO order. A queue's dequeue removes the earliest inserted element that remains, giving FIFO order. The rule determines the element to delete; neither operation accepts an arbitrary key or position.

For an array stack of fixed capacity $m \geq 1$, use zero-based indices and let top be the number of live elements. The data occupy $S[0..top-1]$, with $0 \leq top \leq m$.

```
push(x): if top==m: error overflow
         S[top]=x; top++
pop():   if top==0: error underflow
         top--; return S[top]
```

Checking before push prevents an out-of-bounds write; checking before pop prevents top from becoming −1. By induction, each operation extends or shortens the live prefix, whose last value is the latest push, so LIFO holds. Old bits may remain after pop, but they are outside the logical stack. Clearing references is a separate memory-management concern.

Circular-queue representation invariant

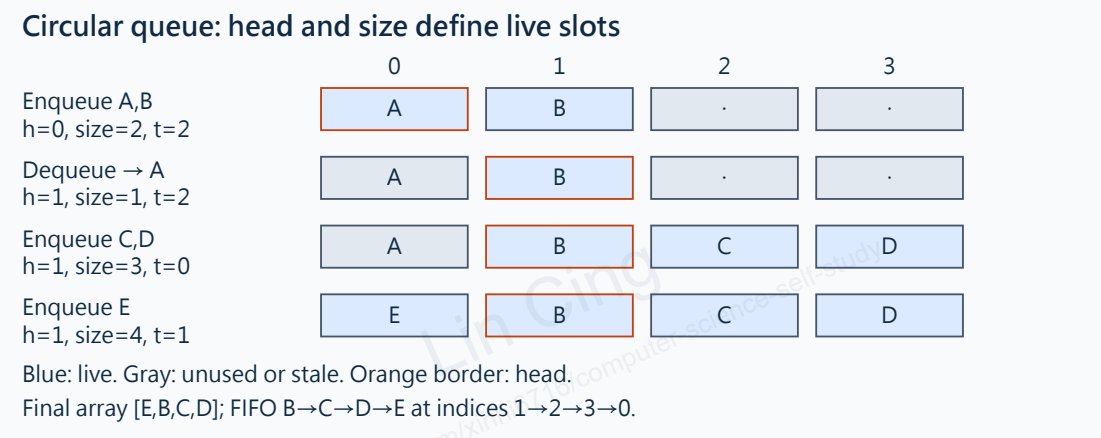
Use $Q[0..m-1]$, head, and size. Define tail as the next write position, $tail=(head+size) \bmod m$. Logical element j is at $Q[(head+j) \bmod m]$, for $0 \leq j < size$.

```
enqueue(x): if size==m: error overflow
            Q[(head+size) mod m]=x; size++
dequeue():  if size==0: error underflow
            x=Q[head]; head=(head+1) mod m; size--; return x
```

Enqueue appends to the old logical sequence. Dequeue returns its old element 0 and advances head once. Reduction modulo m handles wraparound, preserving FIFO order and the indexing invariant. Both size=0 and size=m may have head=tail; size distinguishes them.

The textbook also presents a version that reserves one slot. With only head and tail, empty means head=tail and full means $(tail+1) \bmod m=head$, so usable capacity is $m-1$. Our size-based version holds m elements. Do not mix their fullness tests.

Wraparound preserves FIFO order



At capacity 4, enqueue A,B; dequeue returns A; enqueue C,D; then enqueue E. The final physical array is [E,B,C,D], with head=1, size=4, tail=1. Logical order is B→C→D→E at indices 1→2→3→0. Another enqueue overflows; the next dequeue returns B.

Wraparound preserves FIFO order (continued)

Fixed-array push, pop, enqueue, and dequeue each take worst-case O(1), assuming an element can be moved in O(1). Creating an m-slot array requires Θ(m) space and the corresponding allocation. Automatic growth may move Θ(m) elements in one operation; Chapter 17's amortized analysis is then needed instead of the fixed-capacity per-operation bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linked lists

Array indices determine adjacency in an array; next links determine logical order in a linked list. Adjacent nodes need not occupy adjacent memory addresses. Singly linked nodes store next; doubly linked nodes also store prev. Searching an unsorted list may inspect all n records, taking Θ(n). A local splice with a valid node handle takes O(1).

Without a sentinel, the first node has prev=NIL and the last has next=NIL; deletion must update head or tail separately. A circular doubly linked list has a real dummy sentinel N. N.next points to the first node, N.prev to the last, and both ends link back to N. An empty list has N.next=N.prev=N.

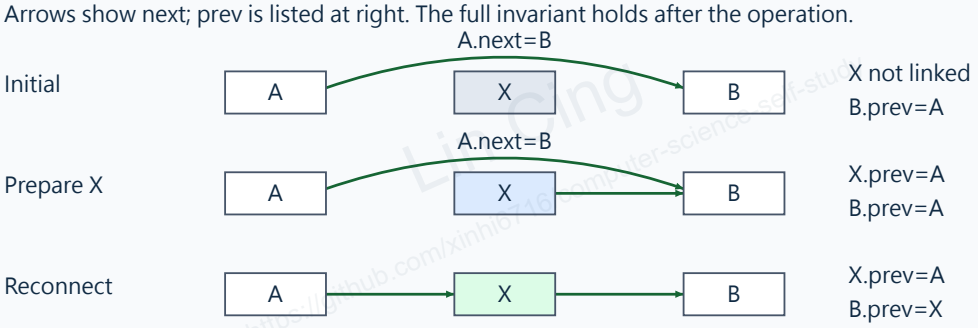
The representation invariant says that following next from N back to N visits every data node exactly once, and every edge u.next=v has v.prev=u. N represents no data key. Search fails on reaching N, and delete must never remove N.

Save the old neighbor before inserting

To insert a new X, currently in no list, between adjacent A and B, first save B=A.next. Write X.prev=A and X.next=B, then A.next=X and B.prev=X. Read an old neighbor before overwriting the link used to find it.

Insertion: save B, then prepare both links of X

Arrows show next; prev is listed at right. The full invariant holds after the operation.



Gray X: uninitialized. Blue: links prepared. Green: reachable from A. A/B may be sentinel.

Save the old neighbor before inserting (continued)

Intermediate writes need not preserve the full doubly linked invariant. This specification assumes a single thread and restores the invariant at the end of the operation. Concurrent readers or exceptions during an operation require additional synchronization or transaction rules; these four writes are not atomic.

The final links are A.next=X, X.prev=A, X.next=B, and B.prev=X. All other edges are unchanged, so the global cycle and reverse-link consistency remain valid. A or B may be the sentinel. The same code handles insertion at either end and insertion into an empty list, where A=B=N.

Deletion preconditions, boundaries, and cost

For a valid data node x, save a=x.prev and b=x.next, then write a.next=b and b.prev=a. The path a→x→b becomes a→b, with the reverse link updated and every other edge unchanged. With one data node, a=b=N, and the two writes restore N's self-loop.

Deleting A from N↔A↔B↔N leaves N↔B↔N. B remains, so the list is not empty. Setting x.next and x.prev to NIL can help debugging, but it does not invalidate external stale handles or replace correct lifetime rules.

O(1) deletion assumes that the caller supplies a valid handle to a node currently in this list and not already deleted. A key alone requires search first, with worst-case Θ(n). Scanning from the head to validate membership on every call also takes Θ(n); that cost cannot be omitted from a claim about the complete operation.

A singly linked list can delete a known predecessor's successor in O(1). Knowing x alone is generally insufficient, particularly at the tail, where copying a successor is impossible. Copying a successor also changes identity and does not implement arbitrary-node deletion under ordinary handle semantics.

Implementing pointers and objects

Slot indices can serve as handles without native pointers. Parallel arrays key[0..m−1], next, and prev use the same index i for one object. Let NIL=−1 so that it cannot be confused with valid slot 0.

For example, an active list may be slot0→slot2 with key[0]=10, key[2]=30, next[0]=2, and prev[2]=0. The logical successor is next[0]=2, not physically adjacent slot1.

In a single-array layout, a fixed three-field object has base=3i, key=A[base], next=A[base+1], and prev=A[base+2]. Specify whether stored pointers are slot indices i or base addresses. Here fields still contain slot indices, so accessing the next object requires multiplication by 3 again. A representation storing base addresses directly must not multiply again.

The two layouts preserve the same list semantics. Variable-length objects do not fit the simple 3i mapping. They require additional block-allocation and length information, so the following fixed-slot free list cannot be applied unchanged.

A free list is a stack of unallocated slots

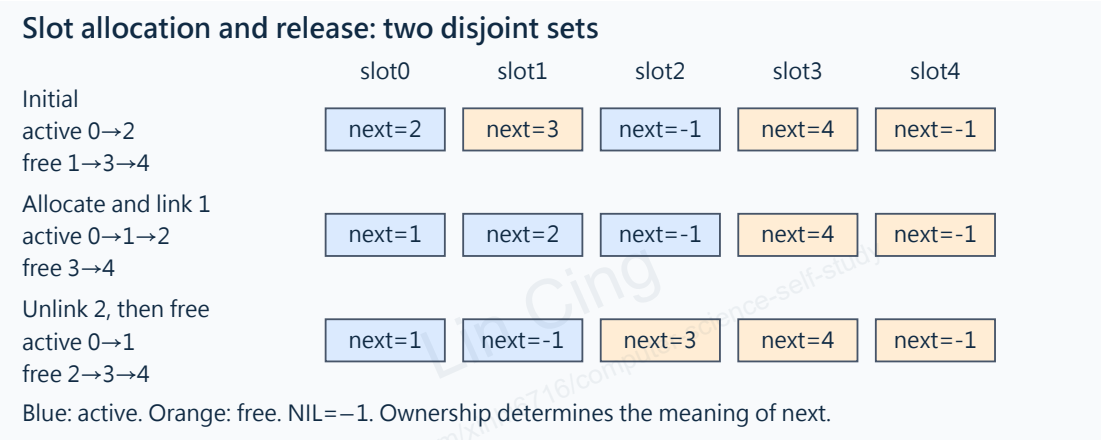
free points to the first unallocated slot, or NIL when none remains. A free slot's next field is reused as its free-list link. Allocation removes the first slot; release prepends a slot that has already been unlinked.

```
allocate(): if free==NIL: error out-of-space
            x=free; free=next[x]; return x
free-object(x): next[x]=free; free=x
```

Allocated and free slots form disjoint sets whose union is all m slots. The free list has no cycle or duplicate. Allocation moves one slot from free to allocated; the caller initializes key, prev, and next before linking it into a data structure. Before release, x must be removed from every structure holding it and must not already be free. Otherwise an active link may be corrupted or a free-list self-loop created.

The free list reads only next, so a free slot's prev and key need not be zeroed. On reuse, initialize every needed field before reading it. This allocator takes O(1), excluding search, destruction of large satellite data, or removal of all external references.

One allocation, insertion, unlinking, and release trace



Initially active is 0→2 and free is 1→3→4. Allocate returns 1 and changes free to 3. Initialize slot1 and insert it between 0 and 2, giving active 0→1→2. Unlink 2 from active before calling free-object(2). The result is active 0→1 and free 2→3→4.

One allocation, insertion, unlinking, and release trace (continued)

If free(2) changes next[2] while active still points to 2, the next traversal enters free slots. Reallocating the slot to another object does not make old handle 2 distinguish the old object from the new one. Do not use a handle after release. Detecting stale handles can use (slot,generation) with version validation, but this adds a representation requirement.

Compacting objects can improve locality but changes index handles. Movement is valid only if all references are updated or a stable indirection table resolves locations. Identical object contents do not make links to its old position valid.

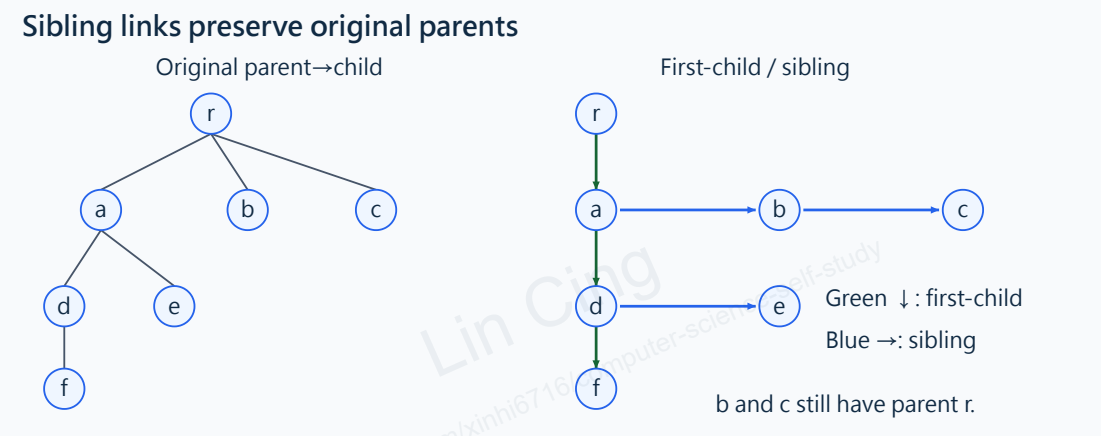
Representing rooted trees

A binary-tree node may store parent, left, and right. The root has parent=NIL, missing children are NIL, and an empty tree has root=NIL. In a general rooted tree, every nonroot still has exactly one parent. The difference is an unrestricted number of children, not permission to have multiple parents.

The first-child/right-sibling representation stores first, sibling, and optionally parent per node. first[x] is x's first child; sibling[x] is the next child of the same parent. The last sibling has sibling=NIL, a leaf has first=NIL, and the root has no sibling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Representing rooted trees (continued)



Here r has a,b,c; a has d,e; and d has f. Thus first[r]=a; sib[a]=b; sib[b]=c; first[a]=d; sib[d]=e; first[d]=f. A sibling edge is not an original parent-child edge. In particular, b's original parent remains r, not a.

Recovering traversal and operation costs

```
visit-tree(x):
    if x==NIL: return
    output key[x]
    c=first[x]
    while c!=NIL:
        visit-tree(c)
        c=sibling[c]
```

Each node is output once. Each parent's loop lists each of its children once, and the whole tree has n−1 children counted across parents. Time is Θ(n), not n nodes multiplied by an independent worst-case n. Stack depth depends on the original tree height h and is O(h+1). A chain needs Θ(n), so O(log n) is not unconditional. This example's preorder is r,a,d,f,e,b,c.

Enumerating all children of x takes Θ(deg(x)+1); a stored parent field gives O(1) ascent. With only parent links, finding all children may require scanning every node. Match representation to operations. A heap uses complete-tree array indices to avoid pointers, whereas a disjoint-set structure can store only parents because it walks toward roots.

Recovering traversal and operation costs (continued)

Mechanically treating first and sibling as binary left and right links also visits the starting node's siblings. That traverses the sibling forest starting at x, which need not equal the original subtree rooted at x. Explicitly enumerating children as above preserves the correct original subtree boundary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 10.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study